```
builder.Services.AddTransient<ICityService, SimpleCityService>();
builder.Services.AddTransient<ICityService, CityService>();
```

This raises an obvious question: which one will get resolved when its abstraction is injected into a constructor? The answer to that question is the last one you registered. So another question arises: how is the capability to inject multiple implementations useful?

Imagine you have several different implementations of a service, but you rely on runtime data to determine which implementation to use. For example, you might want to calculate prices, taxes, and discounts based on the visitor's location. You could fill one service with conditional code for each location that you serve, but you can imagine that approach getting very messy very quickly, especially if the calculations are complex. And you can also imagine the maintenance issues if you need to update the code to reflect changes in laws in one territory, for example. That opens up the possibility of inadvertently changing code for other locations and introducing bugs unrelated to the change you needed to make.

Instead, you could provide a separate implementation for each location. Consider the following simple interface: `IPriceService`.

Listing 7.12 The `IPriceService` interface

```
public interface IPriceService
{
    string GetLocation();
    double CalculatePrice();
}
```

This interface defines two methods—one that returns the location that applies to any specific implementation and another that represents the logic for calculating prices. Let's assume each implementation of this service definition returns the ISO 3166-1 Alpha-2 code you already know about,